Reviewer Pack — Minimal Number of Diophantine Equations and DPLL Proof Tree …

Entry df0da9a97a3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 05:09:13 UTC

Minimal Number of Diophantine Equations and DPLL Proof Tree Width

Entry ID: `df0da9a97a3d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 05:09:13 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

> For any Boolean satisfiability instance with n variables, the number of distinct diophantine equations in its representation is upper-bounded by a function f(n) = O(√n), and this bound is tight for all instances with n ≤ 40.

Rationale (proposer's reasoning):

> Diophantine equations have been used to represent certain complexity classes (e.g., PSPACE). The conjecture suggests that the number of such equations in a SAT instance may provide insights into its DPLL proof tree width, which is a measure of the complexity of solving the instance.

Taxonomy category: `DiophantineRepresentation` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2c2e2581d1f9a13d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the average number of distinct diophantine equations for n ≤ 40 variables across 30 random seeds is less than or equal to √n, and there are no instances with more than 10 equations. The conjecture is falsified if any seed produces an instance with more than 10 equations or if the average number of equations exceeds √n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (1): - `DPLL proof tree width AND upper bound ON number of equations`

Top relevant hits considered: - [http://arxiv.org/abs/cs/0406024v1] Layout of Graphs with Bounded Tree-Width - [http://arxiv.org/abs/0804.4584v1] Feature Unification in TAG Derivation Trees - [http://arxiv.org/abs/1502.02753v4] Ideal Tree-drawings of Approximately Optimal Width (And Small Height)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_sat_instance(n):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(random.randint(2 * n, 4 * n)):
            clause = [random.choice(variables) if random.choice([True, False]) else -random.choice
            clauses.append(clause)
        return clauses

    def diophantine_representation(clauses):
        equations = set()
        for clause in clauses:
            equation = 0
            for literal in clause:
                if literal > 0:
                    equation += literal
                else:
                    equation -= literal
            equations.add(equation)
        return equations

    n_values = [5, 10, 15, 20, 30, 40]
    total_equations = 0
    instances_tested = 0
    max_n = -1
```

```
    for n in n_values:
        for _ in range(5):
            clauses = generate_random_sat_instance(n)
            equations = diophantine_representation(clauses)
            total_equations += len(equations)
            instances_tested += 1
            if len(equations) > 10:
                return {
                    "metric_name": "num_distinct_diophantine_eqs",
                    "metric_value": len(equations),
                    "instances_tested": instances_tested,
                    "n_max": max_n,
                    "conjecture_holds": False,
                    "counterexample": f"Instance with {len(equations)} equations"
                }
            max_n = max(max_n, n)

    mean_equations = total_equations / instances_tested
    return {
        "metric_name": "num_distinct_diophantine_eqs",
        "metric_value": mean_equations,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": mean_equations <= math.sqrt(max_n),
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(seed) for seed in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
re_holds': False, 'counterexample': 'Instance with 15 equations'}
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 13, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 16, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 14, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 15, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 19, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 14, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 16, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 11, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 19, 'instances_tested': 6, 'n
TRIAL: {'metric_name': 'num_distinct_diophantine_eqs', 'metric_value': 17, 'instances_tested': 6, 'n
RESULT: FALSIFIED counterexample="Instance with 17 equations" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test only considers up to n = 40, which is too small to confirm the conjecture's validity for all instances with n ≤ 40. The metric does not scale trivially with n, but the test has not been run on a sufficiently large range of n to support the claim that the bound is tight.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test has produced a counterexample with an instance having 17 distinct diophantine equations, which exceeds the conjectured upper bound of √n for | next: Further investigation is needed to determine if the conjecture holds for all instances with n ≤ 40. Extend the testing range and ensure that no counterexamples exceed the conjectured bound.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14258
2	preregistration	ollama_remote	glm4:latest	0	0	9515
3	novelty	ollama_remote	glm4:latest	0	0	8466
4	novelty	ollama_remote	glm4:latest	0	0	9714
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15189
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11845
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12218
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9859
9	critic	ollama_remote	glm4:latest	0	0	21818
10	judge	ollama_remote	glm4:latest	0	0	13919

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 126800 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/df0da9a97a3d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/df0da9a97a3d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/df0da9a97a3d.tar.gz` (if generated)